Openclaw 炒股助手app

OpenClaw 炒股助手App 开发手册

一、项目概述

产品定位：A股/港股/美股投资者的智能炒股助手，绑定用户OpenClaw账号，提供行情查询、AI分析、持仓管理、智能预警等功能，作为券商开户赠送的增值服务。

核心卖点："开户就送能聊天炒股的AI助手"

技术栈：

- 前端：SwiftUI（iOS 15.0+）
- 后端：FastAPI（Python 3.10+）
- 行情数据源：东方财富API
- AI能力：OpenClaw WebSocket
- 本地存储：Core Data + Keychain

二、文件结构（当前版本）

代码块

```
StockApp/
├── StockApp.xcodeproj          ✅ XcodeGen 生成
├── project.yml                 ✅ 项目配置
├── Resources/Assets.xcassets ✅ 资源文件
├── Services/                   ✅ 核心服务层
│   ├── StockAPIService.swift     — 东方财富实时行情API
│   ├── OpenClawService.swift     — OpenClaw WebSocket 连接
│   ├── LocalStorageService.swift — 本地持久化（自选/持仓）
│   └── AlertService.swift        — 价格预警 + 本地推送
├── Sources/
│   ├── App/StockApp.swift        — @main + 6 Tab 导航
│   ├── Theme/AppTheme.swift      — 黑白配色系统
│   ├── Models/
│   │   ├── Stock.swift           — 股票/指数/K线数据模型
│   │   └── AIMessage.swift       — AI对话消息模型
│   ├── ViewModels/
```

```
│   │   ├── MarketViewModel.swift — 行情页VM
│   │   ├── AIChatViewModel.swift — AI对话VM
│   │   └── SettingsViewModel.swift — 设置页VM
│   ├── Components/CommonComponents.swift — 通用UI组件
│   └── Utils/KeychainHelper.swift — Keychain 存储工具
└── Views/
    ├── MarketView.swift      — 行情页
    ├── WatchlistView.swift  — 自选股页
    ├── PositionsView.swift  — 持仓页
    ├── AlertsView.swift     — 预警规则页
    ├── StockDetailView.swift — K线详情页
    ├── AIChatView.swift     — AI对话页
    └── SettingsView.swift   — 设置页
```

三、已完成功能清单

功能	状态	说明
东方财富真实行情接入	✅	支持个股/指数实时行情、涨跌幅、成交量
6 Tab 导航	✅	行情/自选/持仓/预警/AI/我的
自选股管理	✅	搜索添加、本地持久化、删除、分组
持仓管理	✅	成本录入、市值/盈亏计算、持仓汇总
价格预警	✅	规则管理、本地通知推送、触发记录
K线图	✅	Swift Charts实现，支持分时/日K/周K/月K等7种周期
AI对话	✅	OpenClaw WebSocket 连接，未绑定时降级模拟回复
OpenClaw Token 绑定	✅	Settings 页 API Key 输入框 + 验证
黑白主题	✅	AppTheme 统一设计系统，锁定深色模式

四、后端API接口规范

基础地址

- 开发环境：`http://localhost:8000`

- 生产环境：`https://api.openclaw.com/stock`

接口列表

接口	方法	参数	返回
`/health`	GET	无	健康检查
`/api/v1/quotes`	GET	symbols=股票代码（逗号分隔）	批量实时行情
`/api/v1/kline`	GET	symbol=股票代码, period=周期, limit=条数	K线数据
`/api/v1/search`	GET	q=关键词	股票搜索结果
`/v1/chat`	POST	message=消息, context=上下文	AI对话回复

五、开发环境搭建

iOS端

1. 安装Xcode 15+
2. 安装XcodeGen: `brew install xcodegen`
3. 克隆项目: `git clone <仓库地址>`
4. 生成项目: `cd ios && xcodegen generate`
5. 打开 `StockApp.xcodeproj` 运行

后端

1. 安装Python 3.10+
2. 安装依赖: `cd backend && pip install -r requirements.txt`
3. 启动服务: `uvicorn main:app --host 0.0.0.0 --port 8000 --reload`

六、待开发功能列表

高优先级

- [ ] 板块/资讯Tab实现
- [ ] 每日持仓报告推送

- [ ] AI对话流式输出优化
- [ ] 自选股分组管理
- [ ] 实盘账户对接（券商API）

中优先级

- [ ] 社区功能（讨论/观点/晒单）
- [ ] 量化策略回测
- [ ] 北向资金/龙虎榜数据
- [ ] 财报/公告推送
- [ ] 多设备同步

七、设计规范

配色方案（黑白简洁风格）

- 背景色：`#000000`（纯黑）
- 卡片背景：`#1A1A1A`（深灰）
- 主文字：`#FFFFFF`（纯白）
- 次要文字：`#8E8E93`（系统灰）
- 上涨色：`#FF3B30`（红色，A股红涨绿跌）
- 下跌色：`#34C759`（绿色，A股绿跌）
- 强调色：`#0A84FF`（蓝色，按钮/链接）
- 分割线：`#2C2C2E`

字体规范

- 标题：SF Pro Display Bold, 28pt
- 副标题：SF Pro Display Semibold, 20pt
- 正文：SF Pro Text Regular, 16pt
- 数字/行情：SF Mono Bold, 16pt（等宽对齐）
- 小字：SF Pro Text Regular, 13pt

八、部署指南

iOS端

1. 配置Bundle ID: `com.openclaw.stockapp`
2. 配置推送证书
3. Archive 打包上传App Store

后端

1. Docker 镜像构建: `docker build -t openclaw-stock-api .`
2. 容器运行: `docker run -d -p 8000:8000 openclaw-stock-api`
3. 配置Nginx反向代理 + SSL证书

九、已知问题

1. 东方财富API限流：每分钟最多60次请求，本地加缓存
2. AI对话WebSocket重连逻辑需要优化
3. K线图大数据量下性能有待提升
4. 部分机型文字渲染对齐问题

十、版本历史

版本	发布日期	主要更新
v1.0.0	2026-03-25	基础功能上线：行情/自选/持仓/预警/AI对话

十一、新增功能：OpenClaw 一键绑定

1. 核心价值

产品卖点：用户不需要手动输入API Key，一条命令完成App与本地OpenClaw的绑定，相当于把用户电脑上24小时跑的个人AI助手直接装到手机里，是别家App做不到的差异化优势。

核心宣传语：「别家的AI是公用的，我们的AI是你自己的，24小时为你一个人服务，越用越懂你」

2. 绑定流程（30秒完成）

代码块

```
sequenceDiagram
    participant 用户
    participant App
    participant 后端
    participant 用户本地OpenClaw

    用户->>App: 注册/登录
    App->>后端: 请求生成绑定令牌
    后端->>App: 返回bind_token（有效期15分钟，一次性）
    App->>用户: 显示终端命令: `openclaw bind <bind_token>`
    用户->>用户本地OpenClaw: 复制运行命令
    用户本地OpenClaw->>后端: 上报bind_token + 自身OpenClaw实例ID/API地址
    后端->>后端: 验证bind_token有效，关联用户ID与OpenClaw实例
    后端->>App: 推送绑定成功通知
    App->>用户: 绑定完成，进入主界面
```

3. 绑定页面设计

代码块

```
🔗 绑定你的OpenClaw
────────────────────────────────────────

[ ] 我已经安装了OpenClaw

    └─ 点击复制命令 → `openclaw bind 8a3f2d9c...`
    └─ [📋 复制命令] 按钮
    └─ 状态: 等待绑定...

[ ] 我还没装OpenClaw → 跳转安装指南

────────────────────────────────────────

[ ] 我要手动输入API Key → 跳转到原有输入框
```

4. 后端接口规范

接口	方法	参数	返回
`/api/v1/bind/generate`	POST	user_id	bind_token + 有效期（15分钟）
`/api/v1/bind/verify`	POST	bind_token, openclaw_instance_id, openclaw_api_url	绑定结果（成功/失败/过期）

`/api/v1/bind/status`	GET	bind_token	状态：pending/completed/expired

5. 终端命令说明

用户执行的命令：

```
代码块
openclaw bind <bind_token>
```

命令执行逻辑：

1. 读取本地OpenClaw配置，获取API访问凭证
2. 向后端发起绑定请求，上报实例信息
3. 验证通过后，自动在OpenClaw配置中添加App的访问权限
4. 返回成功/失败提示，同步状态到App

6. 绑定后核心功能

第一优先级（上线必做）

功能	效果	价值
专属记忆AI对话	OpenClaw记住用户所有持仓、交易习惯、提问历史、风险偏好，回答完全个性化	别家AI是公共无记忆的，我们的AI是用户专属的，越用越懂用户
数据双向同步	OpenClaw与App的自选股、持仓、笔记、研报自动双向同步，不用两边重复录入	解决多设备数据不同步痛点
24小时后台预警推送	预警规则在用户本地OpenClaw执行（不会被系统杀后台），支持复杂自定义规则，触发后直接推送到App	比普通App的预警更强大、更稳定
本地知识库同步	用户在OpenClaw里存的研报、财报、博主文章、交易体系，App里随时能搜能看	把用户的个人投研库搬到手机上

第二优先级（上线后1个月迭代）

功能	效果	价值

OpenClaw技能一键调用	电脑端所有投研技能（财报分析、量化回测、舆情监控等）都可以在App点按钮调用，结果直接推送	把复杂投研能力搬到手机上，不用守着电脑
自定义工作流推送	用户可配置规则："每天8:30推持仓报告"、"茅台跌3%自动发分析"等，由OpenClaw后台执行推送	完全个性化的投研自动化
跨设备上下文同步	电脑上聊到一半的问题，手机打开继续聊，不用重说上下文	无缝衔接多设备使用场景

长远功能（差异化壁垒）

功能	效果	价值
多Agent协同指挥	用户可以在App直接派任务给不同子Agent："找AI板块利好"、"写算力板块研报"等，结果直接推送	相当于在手机上指挥一个专属投研团队
个性化内容推荐	根据用户持仓、偏好筛选相关资讯、研报、观点，只推真正关心的内容	真正的千人千面投研信息流

7. 安全与合规

- **隐私优势**：所有用户敏感数据（持仓、交易记录、偏好、笔记）全部存在用户自己的OpenClaw本地，后端不存储任何敏感数据，完全符合券商合规要求，用户不用担心数据泄露
- **安全机制**：
  - bind_token 15分钟过期，绑定一次后立即作废
  - 命令只做绑定操作，不读取/上传用户其他数据
  - 绑定后用户可以随时在OpenClaw里取消授权
- **降级方案**：保留原有「手动输入API Key」选项，兼容不会用终端的用户

十二、⚠ 补充：OpenClaw 接口访问架构（关键）

> 本章节解答核心问题：App 如何真正连接到用户的 OpenClaw?

现状问题

问题	位置	影响
后端 `/v1/chat` 是占位符	main.py	AI对话没有真正接入 OpenClaw
iOS 直接连 `ws://localhost:18789`	OpenClawService.swift	手机离开 Mac 就完全断连

OpenClaw Gateway 是 loopback 模式	openclaw.json	外网无法直接访问

正确架构

代码块

```
┌──────────────────────────────────────────────────────────────────────┐
│                          iOS App                                     │
│                (通过 HTTPS 连接到后端)                                │
└──────────────────────────────────────────────────────────────────────┘
                        │
                        │ HTTPS / WSS
                        ▼
┌──────────────────────────────────────────────────────────────────────┐
│                  FastAPI 后端 (本地 Mac 上)                          │
│                                                                      │
│   ┌──────────────────┐     ┌──────────────────────────────────┐      │
│   │ /v1/chat         │────▶│ OpenClawGatewayClient            │      │
│   │ (真实接入)       │     │ WebSocket → localhost:18789      │      │
│   └──────────────────┘     └──────────────────────────────────┘      │
│                                                                      │
│  后端和 OpenClaw 都在同一台 Mac 上:                                  │
│  → 后端用 127.0.0.1:18789 连接 OpenClaw Gateway                      │
└──────────────────────────────────────────────────────────────────────┘
                        ▲
                        │ localhost:18789
                        │ (同一台 Mac，不走外网)
┌──────────────────────────────────────────────────────────────────────┐
│          OpenClaw Gateway (127.0.0.1:18789)                          │
│          bind: loopback (仅本机访问)                                 │
│          auth: token 模式                                            │
└──────────────────────────────────────────────────────────────────────┘
```

结论：后端必须跑在用户的 Mac 上，这样才能用 `127.0.0.1:18789` 连接 OpenClaw Gateway。iOS App 通过局域网 IP (例如 `http://192.168.1.x:8000`) 或内网穿透后的公网地址访问后端。

远程访问方案（用户不在家时）

OpenClaw Gateway 当前是 `loopback` 模式，外网无法直接连。有两种方案：

方案 A：Tailscale VPN（推荐）

Tailscale 会在 Mac 上创建一个特殊的 VPN 网卡，从任何地方都能用内网 IP 访问 Mac。

配置步骤：

1. Mac 安装 Tailscale：`brew install tailscale`
2. 登录并启动：`sudo tailscaled`
3. 开启 Exit Node（可选，让手机也走这个隧道）
4. 在 OpenClaw 配置里开启 Tailscale 模式：

代码块

```
openclaw configure
# Gateway → Tailscale → mode: on
```

开启后，Gateway 会在 Tailscale 内网分配一个地址（例如 `100.x.x.x`），App 通过这个地址连接。

方案 B：Cloudflare Tunnel（备选）

不需要在路由器做任何配置，Cloudflare 会帮你穿透：

1. 安装 cloudflared：`brew install cloudflared`
2. 创建隧道：`cloudflared tunnel --url http://localhost:18789`
3. 得到一个公网地址，例如 `https://xxx.trycloudflare.com`
4. 后端配置这个地址

缺点：隧道可能不稳定，适合开发测试，不适合生产。

后端真实接入 OpenClaw（修复 main.py）

代码块

```
# main.py — 真实连接 OpenClaw WebSocket
import websocket, threading, json, uuid
from datetime import datetime

# OpenClaw Gateway 配置（来自 openclaw.json）
OPENCLAW_WS_URL = "ws://127.0.0.1:18789/api/chat"
OPENCLAW_TOKEN = "d04457c50a9237f2628f0fb8f0a028a28687b582e1d45026"  # 从配置文件读取

# WebSocket 连接池（每个用户一个连接）
ws_pool: dict[str, websocket.WebSocket] = {}
ws_lock = threading.Lock()

```

```python
def get_or_create_ws(session_key: str) -> websocket.WebSocket:
    """获取或创建用户的 OpenClaw WebSocket 连接"""
    with ws_lock:
        if session_key not in ws_pool or ws_pool[session_key].connected is False:
            ws = websocket.WebSocket()
            ws.connect(f"{OPENCLAW_WS_URL}?session_key={session_key}")
            ws_pool[session_key] = ws
        return ws_pool[session_key]

@app.post("/v1/chat", response_model=OpenClawResponse)
def chat(payload: ChatPayload):
    """
    真实的 OpenClaw AI 对话接口
    - 后端复用 WebSocket 连接池
    - 发送消息到 OpenClaw Gateway
    - 返回 AI 回复
    """
    session_key = payload.context.get("session_key") if payload.context else None
    if not session_key:
        raise HTTPException(status_code=401, detail="未绑定 OpenClaw 账号")

    try:
        ws = get_or_create_ws(session_key)
        # 构造 OpenClaw 消息格式
        msg = json.dumps({
            "type": "message",
            "content": payload.message,
            "context": payload.context or {}
        }, ensure_ascii=False)
        ws.send(msg)
        # 接收回复（超时处理）
        resp = ws.recv()
        data = json.loads(resp)
        return OpenClawResponse(
            reply=data.get("reply", ""),
            suggestions=data.get("suggestions", []),
            actions=data.get("actions", []),
            sessionId=session_key
        )
    except Exception as e:
        # 连接失败时清除池中的连接，下次重试会重建
        with ws_lock:
            if session_key in ws_pool:
                del ws_pool[session_key]
        raise HTTPException(status_code=502, detail=f"OpenClaw 连接失败: {e}")
```

iOS OpenClawService 修复方案

当前 `OpenClawService.swift` 直接连 `localhost:18789`，这在手机上完全无法工作。需要改为通过后端 API：

代码块

```
// OpenClawService.swift — 修复后的版本

class OpenClawService: ObservableObject {
    // ⚠️ 原来: ws://localhost:18789 (死路)
    // ✅ 修复后: 通过后端 API 访问 OpenClaw

    private let baseURL: String
    // 开发环境: Mac 局域网 IP, 例如 http://192.168.1.100:8000
    // 生产环境: https://api.openclaw.com/stock

    init() {
        // 自动检测: 优先用局域网 IP (开发时), 失败后用远程地址
        self.baseURL = UserDefaults.standard.string(forKey: "backend_url")
            ?? "http://192.168.31.94:8000"  // 本地 Mac 的局域网 IP
    }

    // 通过后端代理发送消息
    func sendMessage(_ content: String) async throws -> String {
        guard let token = sessionKey else {
            throw OpenClawError.notConnected
        }

        let url = URL(string: "\(baseURL)/v1/chat")!
        var req = URLRequest(url: url)
        req.httpMethod = "POST"
        req.setValue("application/json", forHTTPHeaderField: "Content-Type")
        req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")

        let body: [String: Any] = [
            "message": content,
            "context": ["session_key": token]
        ]
        req.httpBody = try JSONSerialization.data(withJSONObject: body)

        let (data, resp) = try await URLSession.shared.data(for: req)
        guard let httpResp = resp as? HTTPURLResponse,
              httpResp.statusCode == 200 else {
```

```
            throw OpenClawError.notConnected
        }

        let result = try JSONDecoder().decode(OpenClawResponse.self, from: data)
        return result.reply
    }

    // 健康检查：后端是否在线
    func checkConnection() async -> Bool {
        guard let url = URL(string: "\(baseURL)/health") else { return false }
        do {
            let (_, resp) = try await URLSession.shared.data(from: url)
            return (resp as? HTTPURLResponse)?.statusCode == 200
        } catch {
            return false
        }
    }
}
```

关键配置清单

要让这套架构跑起来，需要完成以下配置：

步骤	操作	状态
1	后端 `main.py` 接入真实 OpenClaw WebSocket（见上方代码）	⬜ 待做
2	后端获取 OpenClaw Token（已配置在 openclaw.json）	✅ 已有
3	OpenClaw Gateway token 暴露给后端（环境变量或配置文件）	⬜ 待做
4	iOS `OpenClawService.swift` 改为走后端 API	⬜ 待做
5	后端部署到用户 Mac 本地（`python3 main.py` 或 launchd 自启）	⬜ 待做
6	iOS App 添加后端地址配置页（SettingsView）	⬜ 待做
7	开启 Tailscale 远程访问（可选，生产必做）	⬜ 待做
8	后端添加 `/api/v1/bind/generate` 等绑定接口	⬜ 待做

部署架构总结

环境	后端位置	OpenClaw Gateway	App 连接方式
开发（同一 WiFi）	Mac 本地 `localhost:8000`	Mac 本地 `127.0.0.1:18789`	`http://192.168.x.x:8000`
远程（Tailscale）	Mac 本地	Mac 本地	`https://api.openclaw.com` → Tailscale IP
生产（远程）	Mac 本地自启（launchd）	Mac 本地	Cloudflare Tunnel 穿透

核心逻辑：OpenClaw Gateway 永远在用户 Mac 本地，后端也在同一台 Mac 上，两者通过 127.0.0.1 通信。App 通过后端间接访问 OpenClaw。